

Here are structured study notes based on the audio transcription about choosing databases in a system design interview:

Choosing a Database in System Design Interviews

In system design interviews, selecting the appropriate database is a crucial decision. Databases are broadly categorized into **SQL** and **NoSQL** types, each with distinct strengths and weaknesses that align with specific use cases.

1. Defining SQL vs. NoSQL Databases

Let's start by defining both types:

* **SQL Databases (Structured Query Language)**

- * **Structure:** Table-based. Data is organized into tables with predefined schemas.
- * **Scalability:** Primarily **vertically scalable** (increasing resources of a single server, e.g., CPU, RAM).

* **NoSQL Databases**

- * **Structure:** Can be document stores, key-value stores, graph databases, or wide-column stores. They are schema-less or have flexible schemas.
- * **Scalability:** Primarily **horizontally scalable** (adding more servers to a distributed system).

2. Strengths of SQL Databases

SQL databases are often chosen for their robust data integrity and structured nature:

- * **Relational Model:**

- * Allow easy querying on relationships between data across multiple tables (e.g., using `JOIN` operations).

- * Table relationships are fundamental for effectively organizing and structuring complex, interlinked data.

- * **Well-Structured Data & Reduced Errors:**

- * SQL schemas require the data model and format to be known *before* storing any data.

- * This predefined structure reduces the room for potential errors and ensures data consistency.

- * **ACID Compliance:**

- * Transactions in SQL databases adhere to the ACID properties:

- * **Atomic:** Transactions are treated as a single, indivisible unit. They either fully complete (all statements execute) or completely fail (none execute).

- * **Consistent:** Transactions bring the database from one valid state to another, maintaining data integrity rules.

- * **Isolated:** Concurrent transactions do not interfere with each other; their execution appears as if they ran sequentially.

- * **Durable:** Once a transaction is committed, the changes are permanent and survive system failures.

3. Weaknesses of SQL Databases

Despite their strengths, SQL databases have limitations:

- * **Setup Time & Rigidity:**

- * Columns and tables must be created and defined ahead of time due to the structured schema requirement.

- * This leads to more setup time compared to NoSQL databases.

- * Less effective for storing and querying **unstructured data** where the format is unknown or frequently changing.

- * **Difficult Horizontal Scaling:**

- * Their relational nature makes them inherently challenging to scale horizontally (across multiple servers).

- * **Read-heavy systems:** Can provision multiple read-only replicas, which is straightforward.

- * **Write-heavy systems:** Often the primary option is to vertically scale the database (upgrade the single server), which is generally more expensive than provisioning additional, cheaper servers.

4. Strengths of NoSQL Databases

NoSQL databases offer flexibility and scalability for modern applications:

- * **Flexibility & Simpler Setup:**

- * Do not typically support rigid table relationships (e.g., joins are often handled at the application layer).

- * Data is usually stored in flexible formats like documents (e.g., JSON) or key-value pairs.

- * A better choice for storing **unstructured or semi-structured data** where the format might evolve or be unknown.

- * **Excellent Horizontal Scaling:**

- * Better at handling unstructured data, they can easily **shard** (partition) this data across different data stores.

- * This enables **distributed databases**, making horizontal scaling much easier and cost-effective.

- * Very large amounts of data can be stored without needing a single, expensive, high-capacity server.

5. Weaknesses of NoSQL Databases

The flexibility and scalability of NoSQL databases often come with trade-offs:

- * **Eventual Consistency (in Distributed Setups):**

- * NoSQL databases, especially when designed for distributed use cases (e.g., multiple write shards for the same data partition using peer-to-peer replication), often trade strong consistency for higher availability and partition tolerance.

- * **Explanation:** After a write operation to one shard in a distributed NoSQL cluster, there's a small delay before that update is propagated to all other replicas.

- * **Risk:** During this propagation delay, reading from a different replica might result in accessing **stale data** (data that has not yet received the latest update).

- * This characteristic, where data eventually becomes up-to-date across all replicas, is called **eventual consistency**.

- * **Important Note:** While a distributed NoSQL database typically exhibits eventual consistency, a *single-shard* NoSQL database *can* be strongly consistent. However, to fully leverage the scalability benefits of NoSQL, the database is usually set up as a distributed cluster, leading to eventual consistency.
